

Tienda de Instrumentos Musicales Don Chuy

-Problemática a resolver-

Planteamos la situación de una pequeña tienda de instrumentos musicales donde se nos solicito crear un programa para que el dueño tenga más control en su día a día en su tienda. Enfocado para sus trabajadores los cuales no tienen conocimientos fuera de los instrumentos musicales, por lo tanto, se solicitó un menú muy simple y fácil de navegar.

El dueño amablemente nos solicitó los siguientes requerimientos para su programa:

- a) Dar de alta productos nuevos con la marca, modelo, el precio y una breve descripción para identificar el producto.
- b) Opción de ver todos los productos almacenados a la vez.
- c) Opción de buscar un producto en específico y la habilidad de eliminarlo del sistema.

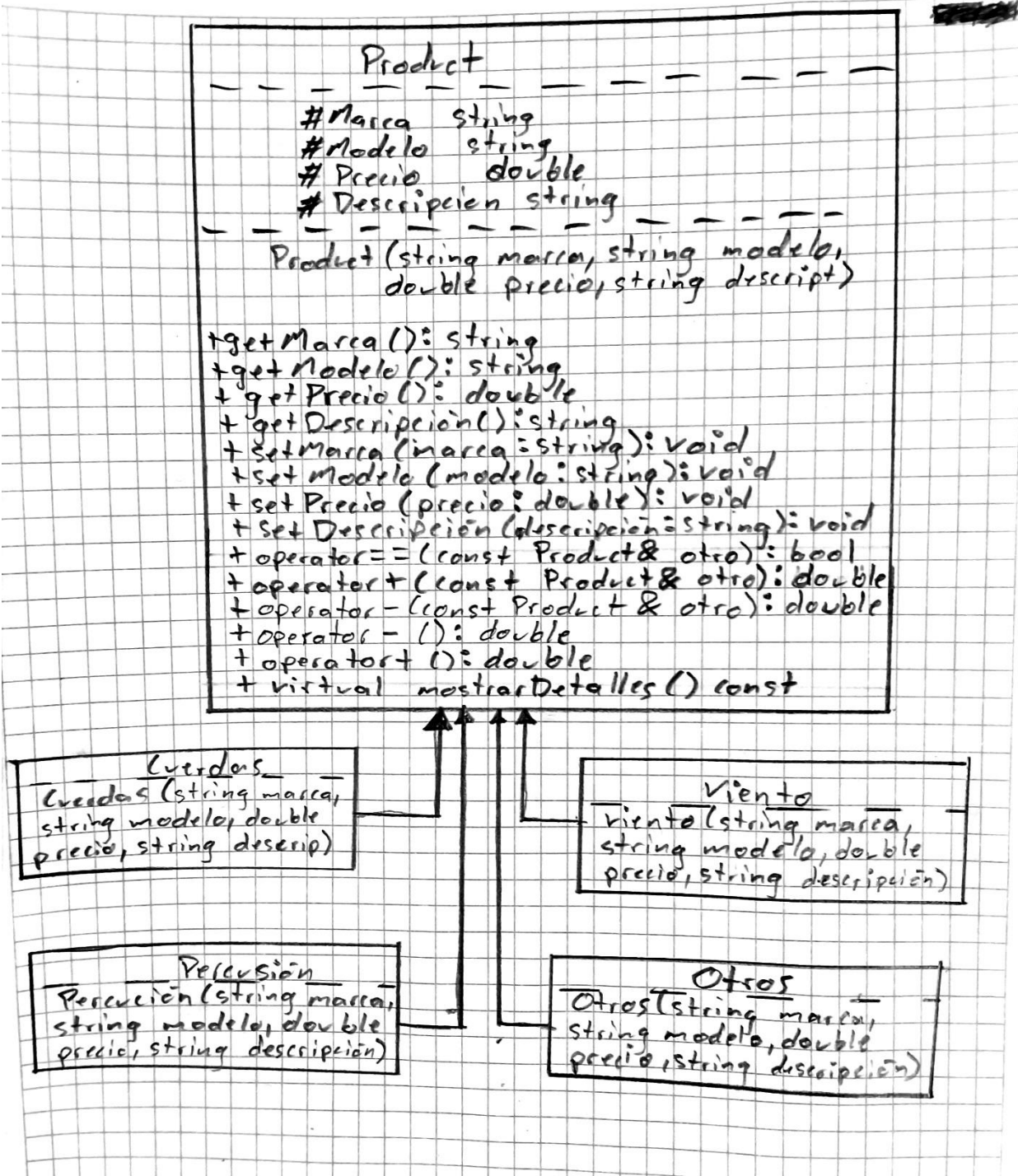
-Requerimientos especiales-

- 1.- Comparar precios de forma rápida entre dos instrumentos
- 2.- Sumar el precio de dos instrumentos
- 3.- Sumar el precio de dos instrumentos con el 15% de descuento
- 4.- Aplicar 5% de descuento en un solo producto en específico
- 5.- Aplicar 5% de incremento de precio en un solo producto en específico

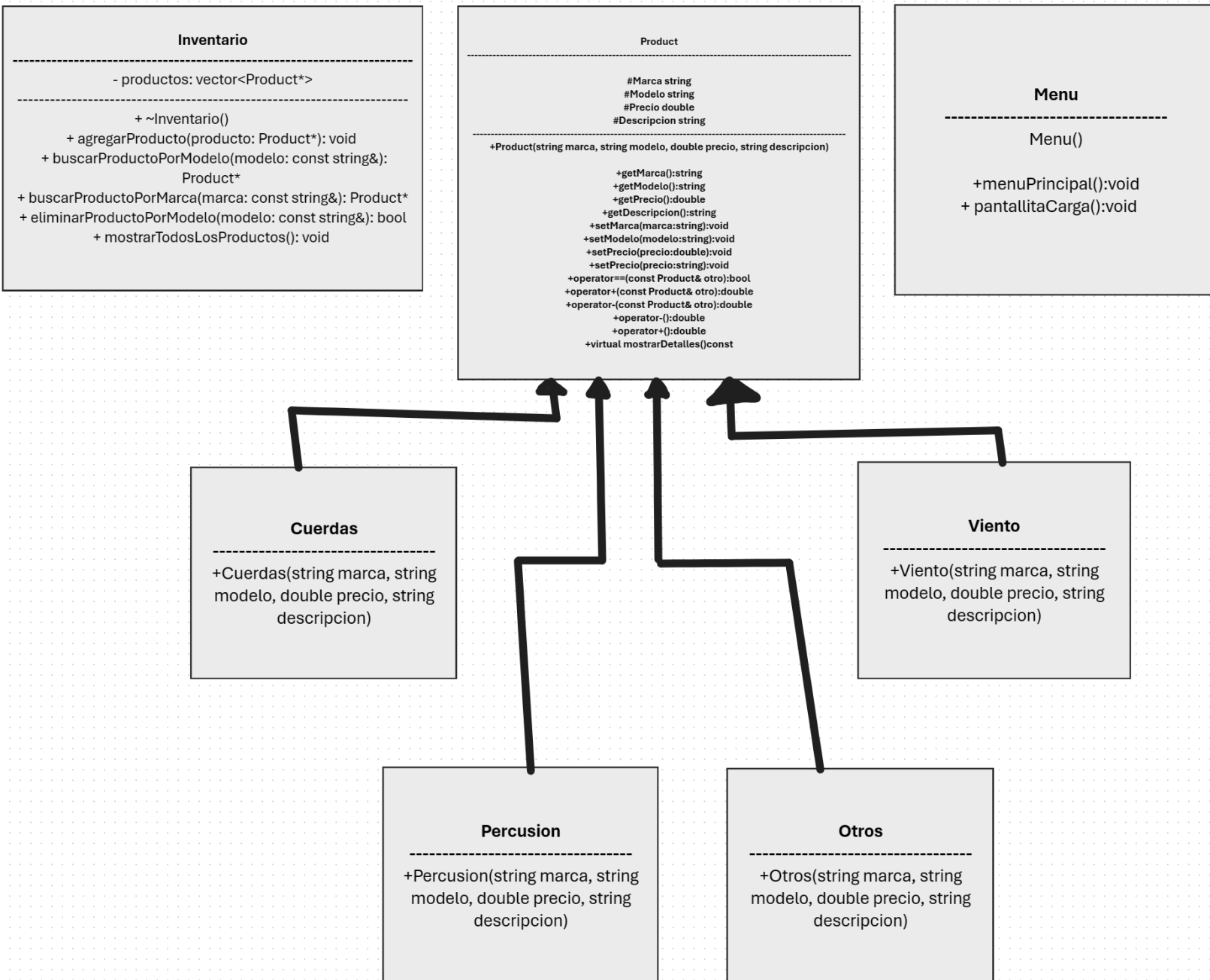
-Implementación realizada con breve descripción-

Diagrama UML-

Primera etapa:



Etapa final:



Descripción breve con apoyo visual y pantallas de carga

Nuestro programa se apoya con el uso de 7 clases y el archivo main, como se señala en el diagrama UML las principales clases son las de "Product", "Inventario" y "Menu", las clases hijas de "Product" son "Cuerdas", "Percusión", "Viento" y "Otros".

A continuación, se explica la lógica de estas clases; clase "Product" es la vertebra de nuestras clases derivadas, aquí con ayuda del archivo .h y .cpp se declara y se implementan los atributos que se heredaran a nuestras clases hijas, aquí se declaran los atributos marca, modelo, precio y descripcion.

Se declara nuestro constructor con parámetros para solicitar por dicha información.

Se declara nuestra función virtual 'mostrarDetalles', por último, en la lógica de este apartado es importante señalar la declaración e implementación de nuestras 'funciones especiales' las cuales son sobrecargas de operadores

```
class Product
{
protected:
    string marca;
    string modelo;
    double precio;
    string descripcion;

public:
    Product(const string& marca = "", const string& modelo = "",
            double precio = 0.0, const string& descripcion = "");

    // Sobrecarga del operador de igualdad para comparar precios
    bool operator==(const Product& otro) const;

    // Sobrecarga del operador +
    double operator+(const Product& otro) const;

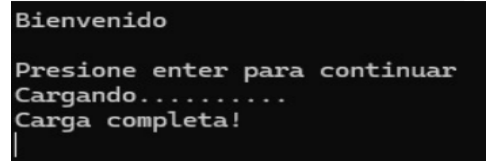
    // Sobrecarga del operador -
    double operator-(const Product& otro) const;

    // Sobrecarga del operador - para aplicar un descuento del 5%
    double operator-() const;

    // Sobrecarga del operador + para aumentar el precio en un 5%
    double operator+() const;
```

Continuando con nuestra clase "Menu". Primera función es 'pantallitaCarga' la cual nos ayuda en controlar mejor el menú y tener una visión más cómoda para el usuario.

La siguiente función es 'menuPrincipal' aquí tenemos toda la implementación de nuestra interfaz para el usuario, esto nos ayuda para que el main no se sature de líneas lo cual ayuda en la gestión del código. Tenemos un menú principal donde podemos: agregar un nuevo producto, buscar el producto existente por el nombre, eliminar un instrumento existente, mostrar todos los instrumentos, salir y por último tenemos acceso al menú de 'funciones especiales'.



```
Bienvenido
Presione enter para continuar
Cargando.....
Carga completa!
```

```
cout << "Elige una opcion:" << endl;
cout << "1- Agregar un nuevo instrumento" << endl;
cout << "2- Buscar instrumento por nombre" << endl;
cout << "3- Eliminar instrumento" << endl;
cout << "4- Mostrar lista de instrumentos" << endl;
cout << "5- Menu de opciones especiales" << endl;
cout << "6- Salir" << endl;
cout << "Su opcion es: ";
cin >> choice;
```

El menú de 'funciones especiales' nos permite: comparar precios, sumar precios con y sin descuento, aplicar 5% de descuento e incremento de un instrumento, por último, claramente tenemos la opción de salir.

```
cout << "Que funcion especial deessas realizar?\n";
cout << "a) Comparar precios de dos productos por sus modelos\n";
cout << "b) Suma sin descuento de dos productos\n";
cout << "c) Suma con descuento de dos productos\n";
cout << "d) Aplicar 5% de descuento en un solo producto\n";
cout << "e) Aplicar 5% de incremento en un solo producto\n";
cout << "f) Salir"<<endl;
cout << "Seleccione una opcion: "; cin >> opcionEspecial;
switch (opcionEspecial)
```

Clase "Inventario"; aquí se destaca la funciona 'agregarProducto' donde se implementa el vector productos y se almacenan los objetos del tipo "Product".

```
void Inventario::agregarProducto(Product* producto)
{
    productos.push_back(producto);
    cout << "Producto agregado exitosamente.\n";
}
```

Aquí se hace uso de memoria dinámica para incrementar el tamaño del vector.

Se implementan las funciones 'buscarProductoPorModelo' y 'buscarProductoPorMarca' para claramente poder buscar los instrumentos de una u otra forma. 'eliminarProductoPorModelo' sirve como indica su nombre para borrar los objetos creados previamente. Por ultimo 'mostrarTodosLosProductos' muestra los objetos creados.

```
public:
    ~Inventario();
    void agregarProducto(Product* producto);
    Product* buscarProductoPorModelo(const string& modelo) const;
    Product* buscarProductoPorMarca(const string& marca) const;
    bool eliminarProductoPorModelo(const string& modelo);
    void mostrarTodosLosProductos() const;
};
```

Aquí principalmente se recalca lo importante y beneficioso de crear estas funciones para separar los 'Problemas' en pequeñas secciones, esto nos ayudo mucho en el momento de reparar los errores y trabajar en conjunto, esto se explicara con mas detalle al final del reporte.

Clases hijas de 'Product': en las clases hijas "Cuerdas", "Percusión", "Viento" y "Otros" tenemos muy poco código, razón de esto fue toda la implementación del código en las clases antes descritas "Product", "Inventario" y "Menu". Todas estas clases claramente

```
class Cuerdas : public Product
{
public:
    Cuerdas(const string& marca, const string& modelo, double precio, const string& descripcion);

    void mostrarDetalles();
};

#endif
```

heredan su código principal de "Product", solo se señala su debido constructor y función 'mostrarDetalles'.

-Conclusion del proyecto y opiniones personales-

Concluimos varias cosas gracias a este proyecto, naturalmente al ser un equipo de dos primeramente se tuvieron que tener en cuenta la disponibilidad de ambas partes lo cual necesito previa visualizacion del proyecto.

En este proceso se penso en un principio en usar el IDE de VisuaStudio Code con una extencion donde ambas partes pudieramos escribir codigo sobre el mismo documento, esto funciono en etapas muy tempranas, despues debido a varios errores con el IDE se desecho esa idea.

Consecuente a lo previo comentado se decidio volver a usar CodeBlocks lo cual ralentizo el proceso y dificulto el estar creando nuevo codigo, en las etapas de probar el avance se tuvo que requerir de erramientas como videollamadas, capturas de pantallas o vernos en persona. Fue de mucha ayuda el diagrama UML inicial donde pensamos en la logica, nombre de variables y funciones.

Por ultimo claramente aprendimos muchas funciones del lenguaje c++ a conforme se presentaban las necesidades y/o retos, Fue un proceso muchas veces de prueba y error, consultar mucho fuentes externas y el libro. No se logro cumplir con todos los requerimientos, sin embargo esto no nos desalento ni nos quito el sentimiento de alivio cuando todas las partes de este puzzle empezaron a encajar.

Muchas gracias por leer y retornos con este proyecto.

Jesus Alejandro Ortiz Silva

223386208

Gabriel Viveros Gutiérrez

223386623